

Software program

The software code for this project is written in 'C' programming language and compiled using the Keil μ Vision3 compiler. After compilation, the final .hex code is downloaded to the microcontroller using a suitable programmer. The source program is well commented and easy to understand.

The main function initialises the timer, ports and LCD. Finally, after enabling the external interrupt 0, it enters into a continuous loop.

Int0 function is an interrupt function and is automatically called when any of the three switches S1 through S3 is pressed.

1. If switch S1 is pressed, it checks if it is pressed for an even/odd number of times. Accordingly, it either switches 'on' or switches 'off' the SCR. Basically, it enables/disables external interrupt 1. The state of the SCR is displayed by a message on the LCD and an indication comes on LED1 and LED2 also.

2. If switch S2 is pressed, the delay is increased by 1 ms and the angle is increased by 18°. The light intensity of the bulb also increases. If the limit is reached, the message is displayed on the LCD.

3. For switch S3, the operation remains the same as with S2, but the delay is decreased by 1 ms and the angle is decreased by 18°.

PARTS LIST

Semiconductors:

IC1	- AT89C51 microcontroller
IC2	- MCT2E opto-coupler
IC3	- 7805, 5V regulator
T1, T2	- BC547 npn transistor
SCR1	- TYN604 SCR
D1-D14	- 1N4007 rectifier diode
LED1-LED5	- 5mm LED

Resistors (all 1/4-watt, $\pm 5\%$ carbon):

R1, R4	- 1-kilo-ohm
R2	- 470-ohm
R3	- 4.7-kilo-ohm
R5, R7-R11	- 330-ohm
R6, R12	- 10-kilo-ohm
VR1	- 10-kilo-ohm preset

Capacitors:

C1	- 1 μ F, 25V electrolytic
C2	- 470 μ F, 16V electrolytic
C3	- 0.1 μ F ceramic disk
C4, C5	- 33pF ceramic disk
C6	- 10 μ F, 16V electrolytic

Miscellaneous:

X1	- 230V AC primary to 9V, 500mA secondary transformer
S1-S4	- push-to-on switch
X _{cr}	- 12MHz crystal
LCD Module	- 16 $\times$ 2 line LCD

Int1 function is also an interrupt function and is automatically called when the zero-crossing detector gives the pulse after every 10 ms. It feeds one pulse to the gate of the SCR after the desired delay (set by switch S2 and S3). The pulse applied is indicated on LED1.

writemcd function sends the command byte to the LCD. It takes one argument byte and sends it to

port P1

writedata function sends data bytes to be displayed on the LCD. It also takes one argument byte and sends it to port P1.

writestr function writes a whole string (message) on the LCD. It takes the pointer as an argument that points the address of the first character of the string. Then through the pointer, it sends all the characters, one by one, to port P0.

busy function checks the status of the busy flag of the LCD. If the flag is set, it means the LCD is not ready and the programs remain within the loop. When the flag is reset, it means the LCD is ready and the program comes out of the loop.

keydly function, used for key debouncing, is the fix delay by approximately 100 ms.

delay function is a variable delay generated by timer 0. The basic delay is of 1 ms, which is rotated in the loop from 1 to 9 times to generate a minimum of 1 ms and a maximum of 9 ms delay.

display function separates each digit of the angle and converts them into an equivalent ASCII number, before sending it to the LCD, one by one, for display.

EFY note. The source code of this project has been included in this month's EFY-CD.

SCR.C

```
#include <reg51.h>
#include <string.h>
sbit rs = P2^6;
// rs pin of LCD
sbit en = P2^7;
// en pin of LCD
sbit rw = P2^5;
// rw pin of LCD
sbit b = P0^7;
// busy flag
sbit led1 = P2^1;
sbit led2 = P2^2;
sbit led3 = P2^3;
sbit led4 = P2^4;
sbit pulse = P2^0;

unsigned int d1=0;
unsigned int d2=0;
unsigned int c=0;

void writemcd(unsigned char a);
// function initializations

void writedata(unsigned char b);
void busy(void);
void writestr(unsigned char *s);
void delay(void);
void decangle(void);
void delay(int d);
void display(unsigned int z);

void keydly(void)
//key debounce delay
{
    unsigned int x,y;
    for(x=0;x<100;x++)
        for(y=0;y<100;y++);
}

void decangle()
//decrease delay by 1ms
{
    EX1=0;
    //first disable all interrupts
    led3=1;

    // indication on LED
    writemcd(0xC0);
    // set next line of LCD
    if(d1>0)
    {
        d1--;
        // decrease delay
        d2-=18;
        // decrease display angle by 18
        writestr("angle:");
        // show it on LCD
        display(d2);
        writestr(" deg. ");
    }
    // display message
    else if(d1==0) writestr("Min
phase angle "); // if max limit is
reached
    led3=0;
    // indication OFF
    EX1=1;
}
```